

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное
бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по учебной практике

ПМ.02. Осуществление интеграции программных модулей

Студент

Стариков Дмитрий Алексеевич

Группа 23П-2

Специальность 09.02.07 Информационные
системы и программирование

Руководитель практики от колледжа:

Калинин Арсений Олегович

Руководитель практики от организации:

_____ Калинин Арсений Олегович

подпись

УТВЕРЖДАЮ:

Директор

Наименование организации

подпись

расшифровка

М. П.

2025-2026 уч. год

Оглавление

- Введение
- Анализ требований, проектирование UX/UI, структуры данных и базы данных
- Разработка REST API 10
 - Создание серверного приложения
 - Контроллеры API
 - Документирование API (Postman коллекция)
- Разработка модуля технолога (WPF desktop)
 - Общие сведения
 - Функциональные возможности
 - Реализация CAPTCHA
 - Взаимодействие с API
- Разработка модуля лаборатории (WPF desktop)
 - Общие сведения
 - Бизнес-функции
 - Автоматическая проверка нормативов
 - Формирование протоколов
 - История испытаний и аудит
- Разработка модуля аппаратчика (WPF desktop)
 - Общие сведения
 - Экран активных партий
 - Программа партии
 - Телеметрия оборудования
 - Фиксация отклонений
- Комплексное тестирование и сопроводительная документации
- Итоговое содержание практики и оформление Git-репозитория
- Заключение
- Приложение

Введение

Целью учебной практики являлась разработка комплексной информационной системы «АгроКонтроль» для автоматизации процессов производства средств защиты растений. Система охватывает три ключевые роли: технолог (разработка рецептов и технологических карт, формирование заказов и партий), лаборант (контроль качества сырья и готовой продукции) и аппаратчик (пошаговое выполнение технологического процесса). В качестве платформы использованы .NET Framework 4.8, WPF для десктоп-приложений, ASP.NET Web API 2 для серверной части и Microsoft SQL Server для базы данных.

В ходе практики выполнены следующие этапы:

Анализ предметной области, проектирование пользовательского интерфейса (wireframe) и логической модели данных.

Разработка базы данных agro_control с хранимыми процедурами, триггерами и функциями.

Создание REST API, обеспечивающего единый слой доступа к данным.

Разработка трёх десктоп-приложений (модули технолога, лаборатории, аппаратчика).

Комплексное тестирование (ручное, модульное, интеграционное) и подготовка сопроводительной документации.

Оформление Git-репозитория согласно требованиям.

Все исходные коды, скрипты и документы размещены в репозитории GitHub (ссылка приведена в README).

Анализ требований, проектирование UX/UI, структуры данных и базы данных

2.1 Анализ предметной области и исходных данных

Предприятие производит агрохимическую продукцию: гербициды, инсектициды, фунгициды, регуляторы роста. Основные бизнес-процессы:

Разработка и версионирование рецептов – технолог создаёт рецепт, указывает компоненты (сырьё) и их количество. Рецепт может иметь несколько версий, одна из которых активна.

Создание технологических карт – для каждого рецепта разрабатывается последовательность шагов (смешивание, выдержка, экструзия, охлаждение) с плановыми параметрами: температура, давление, длительность. Технолог может добавить текстовую инструкцию для аппаратчика.

Планирование производства – менеджер формирует производственные заказы (продукт, плановое количество, дата старта).

Выполнение партий – оператор (аппаратчик) запускает партию на основе заказа и активного рецепта. В процессе выполнения он фиксирует фактические показатели (температуру, давление, время) и добавляет комментарии.

Лабораторный контроль – лаборант отбирает пробы сырья и готовой продукции, вводит измеренные значения, система автоматически сравнивает их с нормативами (хранятся в JSON-поле `standard_params`). Принимается решение «допустить» или «заблокировать».

Отслеживание отклонений – при превышении допусков система автоматически создаёт запись в `deviation_events`. Аппаратчик может также вручную сообщить о проблеме.

Аудит действий – все значимые операции записываются в таблицу `audit_log` с указанием пользователя, времени, типа действия.

Исходные данные предоставлены в виде текстовых файлов (`batch.txt`, `production_order.txt`, `production_step.txt`, `quality_control.txt`, `recipe.txt`, `users.json`). Эти данные были проанализированы и использованы для проектирования структуры базы данных и тестового наполнения.

2.2 Спецификации прецедентов использования (Use Case)

На основе анализа требований выделены следующие прецеденты:

UC ID	Название	Акторы	Краткое описание
UC-01	Управление рецептурами	Технолог	Создание, редактирование, версионирование рецепта. Указание компонентов и их количества.
UC-02	Разработка техкарты	Технолог	Создание технологической карты для рецепта: добавление шагов с плановыми параметрами и инструкциями.
UC-03	Создание заказа	Менеджер / Технолог	Формирование производственного заказа (продукт, плановое количество, дата старта).
UC-04	Запуск партии	Оператор / Мастер	Выбор заказа, привязка активной версии рецепта, указание используемых партий сырья, старт партии.
UC-05	Выполнение шага	Оператор	Для текущего шага ввод фактических параметров (температура, давление, длительность), комментарий, фиксация отклонений.
UC-06	Лабораторный контроль	Лаборант	Выбор партии (сырьё или готовая продукция), ввод измерений, автоматическая проверка соответствия нормативам, принятие решения (допуск/блокировка).
UC-07	Фиксация отклонений	Система / Оператор	Автоматическое создание события при выходе параметров за допуски; ручное добавление инцидента.
UC-08	Прослеживаемость сырья	Аналитик	Отчёт: какие партии продукции использовали конкретную партию сырья и наоборот.
UC-09	Просмотр телеметрии	Аппаратчик	Отображение текущих показателей оборудования (температура, давление, скорость шнека) в реальном времени.
UC-10	Формирование отчётов	Технолог	Экспорт данных (партии, отклонения, результаты контроля) в Excel.

Рисунок 1 - Спецификации прецедентов

Каждый прецедент детализирован: пред- и постусловия, основной поток, альтернативные потоки, исключения. Например, для UC-06 (лабораторный контроль)

предусмотрено обязательное указание причины при блокировке и запрет допуска без завершённого испытания.

2.3 Проектирование пользовательского интерфейса (Wireframe)

В графическом редакторе Figma созданы макеты экранов для каждого модуля. Основные принципы UX:

Крупные целевые области – кнопки высотой не менее 35 pt, легко нажимаются пальцем (учитывается возможное использование на сенсорных панелях в цеху).

Предзаполнение плановых параметров – при выполнении шага поля фактических значений изначально содержат плановые, оператор только корректирует их.

Цветовая индикация – зелёный цвет для пройденных тестов, красный для отклонений, оранжевый для активного шага.

Минимум переходов – информация о партии, шагах и форме ввода собрана на одном экране.

Экран 1: Дашборд (для менеджера/технолога) – карточки KPI (активные партии, процент брака), список текущих партий, график выполнения заказов.

Экран 2: Список партий (для оператора) – таблица с фильтрами, колонки: номер партии, заказ, статус, плановое/фактическое количество, кнопка «Выполнить шаг».

Экран 3: Выполнение шага – заголовок партии и шага; блок плановых значений; форма ввода факта; чекбокс «Отклонение»; комментарий; кнопки «Завершить шаг», «Приостановить». Внизу – история предыдущих шагов.

Экран 4: Лабораторный контроль – выбор партии по QR-коду или из списка; динамический список параметров с полями «Измерено», «Норма», «Результат» (pass/fail); кнопка «Сохранить и принять решение».

Экран 5: Редактор рецептуры – дерево версий, форма для компонентов (ингредиент, количество), кнопка «Создать новую версию».

Экран 6: Отчёт по прослеживаемости – выбор партии сырья или готовой продукции; граф использования.

Макеты сохранены в Figma (ссылка в репозитории).

2.4 Проектирование структуры данных и архитектуры

Логическая модель данных

На основе исходных файлов разработана логическая модель, включающая следующие сущности и связи:

Продукция (products) – справочник выпускаемой продукции.

Сырьё (raw_materials) – справочник компонентов.

Рецепты (recipes) – версии рецептов, привязаны к продукции.

Состав рецепта (recipe_components) – связь рецепта с сырьём и количеством.

Шаги техкарты (process_steps) – последовательность операций для рецепта с плановыми параметрами.

Заказы (production_orders) – плановые задания на производство.

Партии (batches) – фактически запущенные производственные партии, привязаны к заказу и рецепту.

Шаги партии (batch_steps) – фактическое выполнение шагов с временем начала/окончания, параметрами, комментарием.

Партии сырья (raw_material_batches) – поступления сырья с количеством и статусом.

Использование сырья (batch_raw_materials) – какие партии сырья в каком количестве использованы в производственной партии.

Контроль качества (quality_controls) – результаты испытаний (связь с партией или партией сырья).

Отклонения (deviation_events) – зафиксированные несоответствия.

Пользователи (users) – сотрудники с ролями.

Аудит (audit_log) – журнал действий.

Телеметрия (equipment_telemetry) – показания оборудования.

Связи: один ко многим, многие ко многим через промежуточные таблицы. Обеспечена ссылочная целостность внешними ключами с каскадным удалением (например, удаление партии влечёт удаление её шагов).

Физическая модель (SQL)

База данных создана на Microsoft SQL Server (скрипт script.sql).

Основные таблицы приведены ниже (полный DDL в репозитории):

```
sql
CREATE TABLE users (
  id INT PRIMARY KEY IDENTITY(1,1),
  username NVARCHAR(50) NOT NULL UNIQUE,
  password_hash NVARCHAR(255) NOT NULL,
  full_name NVARCHAR(100) NOT NULL,
  role NVARCHAR(20) NOT NULL CHECK (role IN ('technologist','operator','laboratory','admin',...)),
  ...
);

CREATE TABLE products (...);
CREATE TABLE raw_materials (...);
CREATE TABLE recipes (...);
CREATE TABLE recipe_components (...);
CREATE TABLE process_steps (...);
CREATE TABLE production_orders (...);
CREATE TABLE batches (...);
CREATE TABLE batch_steps (...);
CREATE TABLE raw_material_batches (...);
CREATE TABLE batch_raw_materials (...);
CREATE TABLE quality_controls (...);
CREATE TABLE deviation_events (...);
CREATE TABLE equipment_telemetry (...);
CREATE TABLE audit_log (...);
```

Рисунок 2 - БД

Все таблицы нормализованы до 3НФ. Добавлены индексы для ускорения запросов (например, `idx_batches_status`, `idx_quality_batch`).

Архитектура ПО

Выбрана микросервисная архитектура с ядром на монолите (для небольшого предприятия). Уровни:

Клиентский слой – WPF-приложения (три модуля).

API Gateway – реализован как часть ASP.NET Web API (единая точка входа).

Сервис ядра – контроллеры, бизнес-логика (рецепты, партии, заказы).

Сервис лаборатории – выделен в отдельный контроллер QualityController.

Сервис аппаратчика – OperatorController (активные партии, программа, телеметрия).

Сервис аудита – встроен в API (запись в audit_log).

Использованные паттерны проектирования:

Repository – для абстракции доступа к данным (через ADO.NET, но с параметризованными запросами).

Unit of Work – в хранимых процедурах и транзакциях API.

Observer – триггеры БД и логирование аудита.

State – управление жизненным циклом партии (planned → running → completed/blocked).

Strategy – разные алгоритмы проверки нормативов в зависимости от типа параметра.

CQRS (частично) – разделение запросов (GET) и команд (POST/PUT) на уровне контроллеров.

2.5 Разработка базы данных

База данных agro_control создана с помощью скрипта script.sql, который включает:

Создание всех таблиц с первичными ключами, внешними ключами, ограничениями CHECK.

Вставку тестовых данных из исходных файлов (20+ записей в каждой таблице).

Хранимые процедуры:

sp_create_batch – создание партии с JSON-параметром @raw_material_batches_json.

sp_record_batch_step – фиксация выполнения шага с автоматическим вычислением отклонений.

sp_complete_batch – завершение партии с проверкой лабораторного решения.

sp_add_quality_control – добавление результатов контроля, автоматическая блокировка при fail.

Триггеры:

trg_audit_batch_status – запись в audit_log при изменении статуса партии.

(Также триггеры для других критических таблиц.)

Функции:

get_current_user_id() – возвращает ID пользователя из сессии.

get_batches_by_raw_material_batch() – для прослеживаемости сырья (JSON).

Ролевая модель безопасности – созданы роли technologist_role, operator_role, laboratory_role, admin_role с соответствующими правами.

Обеспечена защита от SQL-инъекций через параметризованные запросы и хранимые процедуры.

2.6 Выгрузка в Git

Все исходные коды (API, три WPF-приложения), скрипты БД, документация и артефакты (UML, ERD, wireframe) размещены в Git-репозитории. Репозиторий содержит следующие ветки и теги, отражающие этапы разработки. Ссылка на репозиторий: [здесь будет ссылка].

Разработка REST API

3.1 Создание серверного приложения

Разработан проект AgroChem.API на ASP.NET Web API 2 (.NET Framework 4.8) с использованием OWIN (Katana). Основные компоненты:

OWIN Startup – конфигурация маршрутов, внедрение зависимостей (DI) через Unity (опционально).

Контроллеры – наследуются от ApiController.

Аутентификация – реализована без JWT по требованию; используется простая проверка логина/пароля с возвратом роли и имени пользователя. CAPTCHA генерируется на клиенте.

Работа с БД – через ADO.NET (SqlConnection, SqlCommand) с параметризацией запросов.

3.2 Контроллеры API

Контроллер	Базовый маршрут	Ключевые методы
AuthController	/api/auth	POST /login, POST /register (возвращает success, role, fullName)
ProductsController	/api/products	GET, POST, PUT, DELETE
RecipesController	/api/recipes	GET, POST, PUT, DELETE; + GET /api/recipes/{id}/components
ProcessStepsController	/api/recipes/{id}/steps	GET, POST, PUT, DELETE
OrdersController	/api/orders	CRUD
BatchesController	/api/batches	CRUD, дополнительно GET /api/batches/active
QualityController	/api/quality	GET /batches?type=raw/final; GET /standards?productId=...; POST /save; GET /history; PUT /update/{id}
OperatorController	/api/operator	GET /active-batches; GET /batch/{id}/program; POST /batch/{id}/step/{order}/start; POST /batch/{id}/step/{order}/complete; GET /telemetry/{equipmentName}
DeviationEventsController	/api/deviations	GET (список отклонений)
ReportsController	/api/reports	GET /batches/excel, GET /deviations/excel (генерируют CSV/Excel)

Рисунок 3 - Контроллеры API

Все контроллеры реализуют асинхронные методы (async/await) и возвращают IActionResult с соответствующими кодами HTTP.

3.3 Документирование API (Postman коллекция)

Создана Postman коллекция AgroChem_API.postman_collection.json, которая включает:

Папку «Auth» – логин и регистрация.

Папку «Products» – все CRUD операции.

Папку «Recipes» – получение рецептов, создание, обновление.

Папку «ProcessSteps» – управление шагами техкарты.

Папку «Orders» – заказы.

Папку «Batches» – партии.

Папку «Quality» – лабораторный контроль, нормативы, история.

Папку «Operator» – активные партии, программа, шаги, телеметрия.

Папку «Reports» – экспорт.

Для каждого эндпоинта приведены примеры запросов и ожидаемых ответов. Коллекция экспортирована в формате JSON и находится в репозитории в папке docs/postman.

Разработка модуля технолога (WPF desktop)

4.1 Общие сведения

Проект AgroChem.TechnologistClient – настольное WPF-приложение (.NET Framework 4.8). Архитектура: MVVM (использованы привязки, команды, но для простоты часть логики вынесена в code-behind). Основные окна:

LoginWindow – авторизация с CAPTCHA.

MainWindow – главное окно с меню и контейнером для UserControl.

ProductsControl – управление продукцией.

RecipesControl – управление рецептурами.

TechMapControl – технологические карты.

OrdersControl – заказы.

BatchesControl – партии.

MonitoringControl – мониторинг выполнения партий.

DeviationsControl – отклонения.

ReportsControl – отчёты.

4.2 Функциональные возможности

Функция	Реализация
Вход с CAPTCHA	Генерация капчи в виде графического изображения (цифры/буквы) с помощью System.Drawing. Пользователь вводит код, проверка на клиенте. После успешного входа роль technologist проверяется на сервере.
Продукция	DataGrid с возможностью добавления, редактирования, удаления. Поля: наименование, тип, форма выпуска, активность.
Рецептуры	Работа с версиями. Можно создать новую версию на основе предыдущей, добавить компоненты (сырьё + количество). Статус: draft, active, archived.
Технологические карты	Для выбранного рецепта – список шагов с возможностью добавления, изменения порядка, указания плановых параметров и инструкции для аппаратчика.
Заказы	Создание заказа с указанием продукта, планового количества, даты старта.
Партии	Создание партии на основе заказа (автоматически подтягивается активный рецепт). Возможность привязать партии сырья.
Мониторинг	Отображение активных партий (статус running) с проценткой выполнения шагов.
Отклонения	Таблица событий из deviation_events с фильтром по партии и серьёзности.
Отчёты	Экспорт списка партий и отклонений в Excel (через Microsoft.Office.Interop.Excel).

Рисунок 4 - Функциональные возможности

4.3 Реализация CAPTCHA

В отдельном классе `CaptchaHelper` генерируется случайный код (6 символов) и `Bitmap` с искажениями (шум, линии). Изображение отображается в `Image` элементе. После ввода пользователем кода выполняется сравнение; при несовпадении – ошибка и генерация новой капчи.

```
csharp
public static (byte[] imageBytes, string code) GenerateCaptcha(int width=200, int
height=80)
{
    string code = GenerateRandomCode(6);
    using (var bitmap = new Bitmap(width, height))
    using (var graphics = Graphics.FromImage(bitmap))
    {
        graphics.Clear(Color.WhiteSmoke);
        using (var font = new Font("Arial", 28, FontStyle.Bold | FontStyle.Italic))
        {
            graphics.DrawString(code, font, Brushes.DarkBlue, ...);
        }
        // добавление шума и линий
        ...
        bitmap.Save(ms, ImageFormat.Png);
        return (ms.ToArray(), code);
    }
}
```

Рисунок 5 - Реализация CAPTCHA

4.4 Взаимодействие с API

Сервис `ApiService` содержит методы для каждого вызова: `GetProductsAsync`, `CreateRecipeAsync`, `GetBatchesAsync` и т.д. Используется `HttpClient` с базовым адресом

https://localhost:44308. Обработка ошибок – через try-catch с показом пользовательского сообщения.

Разработка модуля лаборатории (WPF desktop)

5.1 Общие сведения

Проект AgroChem.Laboratory – WPF-приложение для контроля качества. Архитектура: code-behind с элементами MVVM (свойства INotifyPropertyChanged).

Основные окна:

LoginWindow – вход (аналогичен модулю технолога).

MainWindow – главное окно с переключателем «Сырьё / Готовая продукция», списком партий, кнопками «Контроль качества» и «История».

QualityControlWindow – форма ввода параметров с динамическими полями.

HistoryWindow – просмотр истории испытаний выбранной партии.

EditTestWindow – редактирование уже проведённого испытания.

5.2 Бизнес-функции

Контроль качества сырья:

При выборе вкладки «Сырьё» загружаются партии сырья со статусом available, у которых ещё нет контроля (qc_status = pending).

Лаборант выбирает партию, открывается QualityControlWindow.

Нормативы загружаются из standard_params таблицы raw_materials через API (GET /api/quality/standards?rawMaterialId=...).

Контроль готовой продукции:

Вкладка «Готовая продукция» – партии со статусом running или completed.

Нормативы из products.standard_params.

Фиксация результатов:

Для каждого параметра генерируется строка: название, поле ввода, эталонное значение, результат (pass/fail).

При изменении MeasuredValue вызывается метод EvaluateParameter, который парсит норматив и определяет, попадает ли значение в допуск.

Результат отображается цветом и текстом ( pass /  fail).

Принятие решения:

Кнопка «Допустить» активна только если все параметры введены и результат pass.

Кнопка «Заблокировать» требует обязательного указания причины в поле txtBlockReason.

Решение отправляется на сервер через POST /api/quality/save. При блокировке партии статус batches меняется на blocked.

Формирование протоколов:

Кнопка «Экспорт протокола» генерирует Excel-файл через библиотеку ClosedXML. Файл содержит таблицу: параметр, измеренное значение, норма, результат.

Имя файла: Protocol_<sample_type>_yyyyMMdd_HH:mm:ss.xlsx.

5.3 Автоматическая проверка нормативов

Реализован разбор следующих форматов нормативов:

Диапазон: 6.5-7.0 → $val \geq 6.5 \ \&\& \ val \leq 7.0$

Больше: >97% → $val > 97$

Меньше: <2.5% → $val < 2.5$

Больше или равно: ≥99 → $val \geq 99$

Меньше или равно: ≤0.5 → $val \leq 0.5$

Точное равенство: pass → `string.Equals`

Код функции EvaluateParameter размещён в QualityControlWindow.xaml.cs.

5.4 История испытаний и аудит

Окно HistoryWindow загружает все записи quality_controls для выбранной партии (через GET /api/quality/history?batchId=...).

Для каждой записи отображаются дата, лаборант, решение, параметры (сводка).

Кнопка «Редактировать» открывает EditTestWindow, где можно изменить измеренные значения и комментарий. После сохранения отправляется PUT /api/quality/update/{id}.

Аудит: при сохранении новых результатов или обновлении старых в audit_log записывается действие SAVE_QUALITY или UPDATE_QUALITY с JSON-копией данных.

Разработка модуля аппаратчика (WPF desktop)

6.1 Общие сведения

Проект AgroChem.OperatorClient – WPF-приложение для выполнения технологических операций. Основные окна:

LoginWindow – вход (роль operator).

ActiveBatchesWindow – список активных партий.

BatchProgramWindow – основное окно программы партии.

6.2 Экран активных партий

После входа отображается DataGrid с колонками: номер партии, продукт, текущий шаг, линия, статус. Данные получаются через GET /api/operator/active-batches, который возвращает партии со статусом running. При выборе партии и нажатии «Выбрать партию» открывается BatchProgramWindow с передачей batchId.

6.3 Программа партии

Окно разделено на три части:

Слева – ListBox со всеми шагами техкарты. Для каждого шага отображается название и цветовой индикатор статуса: серый (not_started), оранжевый (in_progress), зелёный (completed).

Центр – детали выбранного шага:

Инструкция (поле instruction из process_steps).

Плановые параметры: температура, давление, длительность (только для чтения).

Форма ввода фактических значений (текстовые поля).

Комментарий (многострочное текстовое поле).

Кнопки «Начать шаг» и «Завершить шаг» (кнопка завершения активна только после начала).

Справа — панель телеметрии оборудования (экструдер). Отображаются температура, давление, скорость шнека и время последнего обновления.

Логика выполнения шага:

Аппаратчик выбирает шаг (автоматически выбирается первый невыполненный).

Нажимает «Начать шаг» → отправляется POST `/api/operator/batch/{id}/step/{order}/start` → в БД фиксируется `actual_start_time`.

Выполняет операцию физически, вводит фактические показатели (можно оставить пустыми).

Нажимает «Завершить шаг» → отправляется POST `/api/operator/batch/{id}/step/{order}/complete` с фактическими параметрами

Сервер вычисляет отклонения (если фактические значения выходят за плановые допуски: температура $>2^{\circ}$, давление >0.5 бар, время >10 мин) и создаёт записи в `deviation_events`.

Если это был последний шаг, партия автоматически переводится в статус `completed`.

6.4 Телеметрия оборудования

На сервере создана таблица `equipment_telemetry`. В демонстрационных целях в ней хранятся текущие показания (можно обновлять через отдельный сервис или вручную). В `BatchProgramWindow` запущен `DispatcherTimer` с интервалом 3 секунды, который вызывает `GET /api/operator/telemetry/Экструдер Линия 1` и обновляет панель.

6.5 Фиксация отклонений

Отклонения фиксируются автоматически при завершении шага, если абсолютная разница между фактическим и плановым значением превышает установленные пороги. Для каждого превышения создаётся запись в `deviation_events` с указанием `batch_id`, `batch_step_id`, типа (`temperature_deviation`, `pressure_deviation`, `duration_deviation`), описания и серьёзности (`low`, `medium`, `high`). Аппаратчик также может вручную добавить

проблему через комментарий (но в текущей реализации комментарий не триггерит событие – это можно доработать).

Комплексное тестирование и сопроводительная документация

7.1 Ручное функциональное тестирование (40 тестов)

Разработаны 40 тест-кейсов (32 позитивных + 8 негативных, 20% негативных), оформленных в виде таблицы Excel. Примеры:

ID	Модуль	Название	Шаги	Ожидаемый результат
ТС-T-01	Технолог	Вход с корректными данными	Ввод логина <code>tech.ivanov</code> , пароля <code>123456</code> , капчи	Открывается главное окно
ТС-T-02	Технолог	Создание продукции	Заполнить форму, сохранить	Новая строка в DataGridView
ТС-T-03 (негативный)	Технолог	Вход с неверным паролем	Ввод неверного пароля	Сообщение «Неверный пароль»
ТС-L-01	Лаборатория	Допуск партии по норме	Ввод корректных параметров	Партия approved, статус обновлён
ТС-L-02 (негативный)	Лаборатория	Блокировка без причины	Нажать «Заблокировать» без текста	Предупреждение «Укажите причину»
ТС-O-01	Аппаратчик	Выполнение шага	Начать шаг, ввести данные, завершить	Шаг отмечен как completed, данные сохранены
ТС-O-02 (негативный)	Аппаратчик	Завершение шага без начала	Нажать «Завершить шаг» без начала	Кнопка неактивна

Рисунок 6 - Ручное функциональное тестирование

Все тесты выполнены, результаты зафиксированы в документе «Тестовая документация.xlsx» в репозитории.

7.2 Модульное тестирование (30 тестов)

Созданы проекты xUnit для каждого модуля desktop:

AgroChem.Tests.Technologist – 10 тестов (валидация рецептов, компонентов).

AgroChem.Tests.Laboratory – 10 тестов (парсинг нормативов, вычисление результата).

AgroChem.Tests.Operator – 10 тестов (обнаружение отклонений по температуре, давлению, времени).

Пример теста для лаборатории:

```
csharp
[Fact]
public void EvaluateParameter_ValueWithinRange_ReturnsPass()
{
    var param = new TestParameter { StandardValue = "6.5-7.0", MeasuredValue = "6.8"
};

    EvaluateParameter(param);
    Assert.Equal("✅ pass", param.Result);
}
```

Рисунок 7 - Пример теста для лаборатории

Тесты запускаются через Обозреватель тестов Visual Studio или консоль dotnet test. Все 30 тестов успешно проходят.

7.3 Интеграционное тестирование API (10 тестов)

Проект AgroChem.Tests.API содержит тесты, которые отправляют реальные HTTP-запросы к запущенному API (адрес <https://localhost:44308>). Проверяются:

POST /api/auth/login – успешный и неуспешный вход.

GET /api/products – получение списка.

GET /api/operator/active-batches – получение активных партий.

POST /api/quality/save – сохранение результатов контроля.

GET /api/quality/standards?productId=1 – получение нормативов.

PUT /api/quality/update/{id} – обновление испытания.

Негативные сценарии (неправильные параметры, несуществующий ID).

Все интеграционные тесты зелёные при запущенном API.

7.4 Пояснительная записка

Подготовлен документ «Пояснительная записка» (формат PDF и DOCX), в котором описаны:

Цели и задачи автоматизации.

Архитектура системы (компоненты, взаимодействие).

Функциональные требования.

Описание разработанных модулей.

Результаты тестирования.

Инструкция по развёртыванию.

7.5 Руководство оператора для модуля технолога

Создано «Руководство оператора» (PDF, DOCX), содержащее:

Установку и запуск приложения.

Авторизацию с САРТСНА.

Работу с продукцией, рецептурами, технологическими картами.

Создание заказов и партий.

Мониторинг выполнения и просмотр отклонений.

Формирование отчётов.

7.6 Документ «Отладка программного модуля»

Для каждого из четырёх компонентов (API, модуль технолога, модуль лаборатории, модуль аппаратчика) подготовлен отдельный документ, в котором отражены:

Цели отладки.

Используемые инструменты (отладчик Visual Studio, SQL Server Profiler, Postman).

Этапы отладки (описание выявленных дефектов и способов их устранения).

Примеры исправлений (например, ошибка открытого DataReader в OperatorController, исправление парсинга нормативов).

Рекомендации по дальнейшей поддержке.

Все документы размещены в папке docs репозитория.

Итоговое содержание практики и оформление Git-репозитория

8.1 Оформление README.md

Файл README.md содержит:

Краткое описание проекта.

Инструкцию по запуску API (через Visual Studio, порт 44308).

Инструкцию по запуску десктоп-приложений.

Учётные данные для тестирования:

Технолог: tech.ivanov / 123456

Лаборант: lab.vasilieva / 123456

Аппаратчик: operator.zavodov / 123456

Список использованных технологий.

Сведения об авторах.

Репозиторий не содержит архивов, все файлы в открытом виде, оригиналы документов дополнены копиями в формате PDF.

Заключение

В результате прохождения учебной практики полностью разработана информационная система «АгроКонтроль», которая автоматизирует ключевые процессы агрохимического предприятия. Система включает:

Надёжную базу данных (MS SQL Server) с поддержкой ссылочной целостности, триггеров и хранимых процедур.

REST API (ASP.NET Web API 2) с 10 контроллерами, обеспечивающими авторизацию, регистрацию, CRUD для сущностей, лабораторный контроль, пошаговое выполнение партий, телеметрию и аудит.

Три десктоп-приложения (WPF) для технолога, лаборанта и аппаратчика, каждое из которых реализует свой набор бизнес-функций с удобным и интуитивно понятным интерфейсом.

Полный комплект тестирования (40 ручных тестов, 30 модульных, 10 интеграционных), подтверждающий корректность работы системы.

Исчерпывающую документацию: пояснительная записка, руководство оператора, документы по отладке.

Все артефакты размещены в Git-репозитории в соответствии с требованиями (отсутствие архивов, наличие копий в PDF, структурированный README). Система готова к внедрению в промышленную эксплуатацию.

ПРИЛОЖЕНИЕ

Qr-Code:

